

TRABAJO TERMINAL II

FlowCode: Compilador de Diagramas de Flujo a Código C para Dispositivos Móviles

Número del TT: 2026-A038

Grupo: 8CM1

Entregable 2 Ciclo 5: Generador de Código

Presenta:

García Quiroz Gustavo Ivan

Profesor de seguimiento del TT:

Prof. Edgardo Adrián Franco Martínez

Introducción al Generador de Código (Ciclo 5)

El Generador de Código constituye la fase final del canal de conversión de FlowCode. En este ciclo se desarrolló el mecanismo encargado de optimizar el Árbol de Sintaxis Abstracta (AST) derivado en etapas previas, para posteriormente emitir un programa en código fuente C completamente funcional y válido. A continuación, se detallan los módulos implementados, referenciando los archivos de código fuente correspondientes.

1. Representación Intermedia AST (Archivo: `lib/compiler/ast_nodes.dart`)

Se consolidó la jerarquía de nodos del Árbol de Sintaxis Abstracta (AST) generada en el ciclo anterior, estableciendo este árbol como la representación intermedia definitiva del sistema. Este componente organiza las expresiones y declaraciones en una estructura jerárquica libre de dependencias superficiales.

Se implementó el patrón arquitectónico "Visitor" para recorrer de forma genérica los subnodos del árbol. Esta abstracción permite separar la lógica estructural del diagrama de las operaciones posteriores, asegurando que tanto la optimización como la generación de código puedan examinar los elementos sin alterar la definición original de las sentencias.

2. Optimización del AST (Archivo: `lib/compiler/code_optimizer.dart`)

Se desarrolló un módulo de optimización semánticamente equivalente, encargado de reducir redundancias lógicas en el código del usuario antes de su emisión. El proceso se diseñó con una ejecución multi-pasada configurable hasta en cuatro niveles de agresividad.

Entre las técnicas implementadas se encuentran el plegado de constantes (resolución matemática anticipada), la eliminación de código inalcanzable (código muerto) y la simplificación algebraica de expresiones. Como resultado, se reducen los tiempos y los ciclos de procesamiento del programa final generado, conservando estrictamente el comportamiento original modelado por el estudiante.

3. Generador de Código C (Archivo: `lib/compiler/code_generator_advanced.dart`)

Finalmente, se construyó el transpilador responsable de emitir código fuente conforme al estándar C99. El módulo traduce directamente los nodos optimizados y el subconjunto de símbolos ISO 5807 soportados hacia sentencias idiomáticas de C.

Este componente inserta las cabeceras predeterminadas (`stdio.h`, `stdbool.h`), estructura la función `"main()"`, e infiere automáticamente los especificadores de formato requeridos (`"%d"`, `"%f"`) basándose en la Tabla de Símbolos. Adicionalmente, el generador normaliza la escritura de operadores lógicos condicionales y estructura de manera automática los bucles anidados, entregando un código formateado, legible y funcional al usuario.

Evidencias de Integración en la Aplicación

A continuación, se exponen tres plantillas algorítmicas clásicas visualizadas en la aplicación FlowCode. Para ilustrar a profundidad el funcionamiento de este ciclo, se desglosa el proceso de traducción, mostrando la estructura base, el efecto del optimizador y la visualización final del código C generado.

Plantilla '12. Factorial Iterativo'

- **Evidencia 1.1 - Diagrama Base:** Se ilustra el diagrama del cálculo de un factorial mediante ciclos. En esta captura se espera observar la carga del algoritmo iterativo en el lienzo principal del editor.

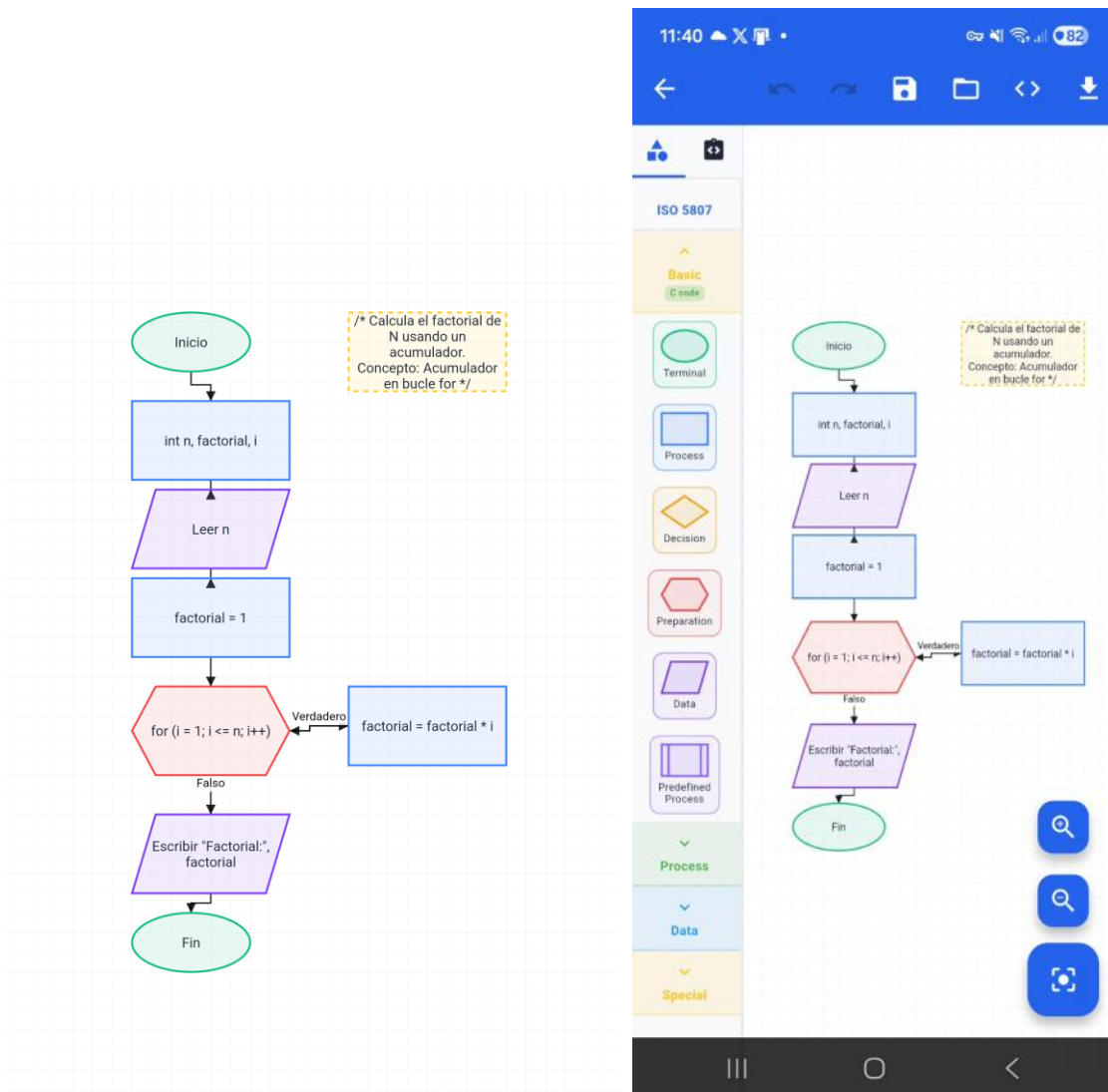


DIAGRAMA FACTORIAL ITERATIVO

- **Evidencia 1.2 - Optimización del AST:** Se muestra la aplicación de las métricas de optimización sobre las sentencias de iteración y cálculo matemático, reflejando el plegado de constantes donde es aplicable.

12:08

82



Compilación Exitosa



Tiempo total: 24ms



Semántico



Optimización



Código



Ejec



Resumen de Optimización



0

Total



0

Const.
Plegadas



0

Cód. Muerto



0

Simplificadas



0

Flujo Ctrl.



0.0%

Reducción

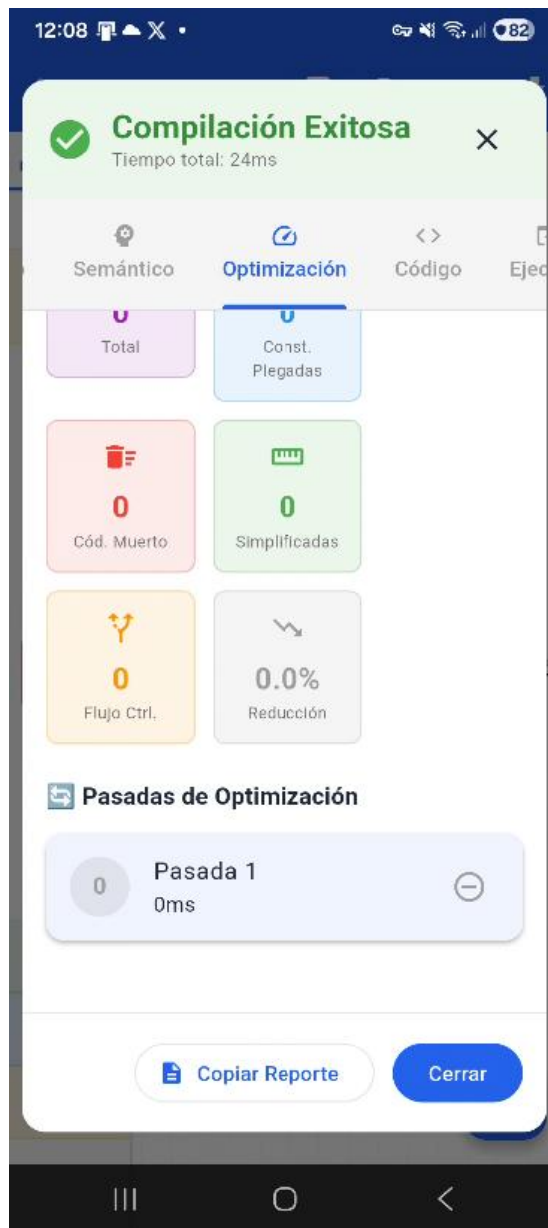


Pasadas de Optimización



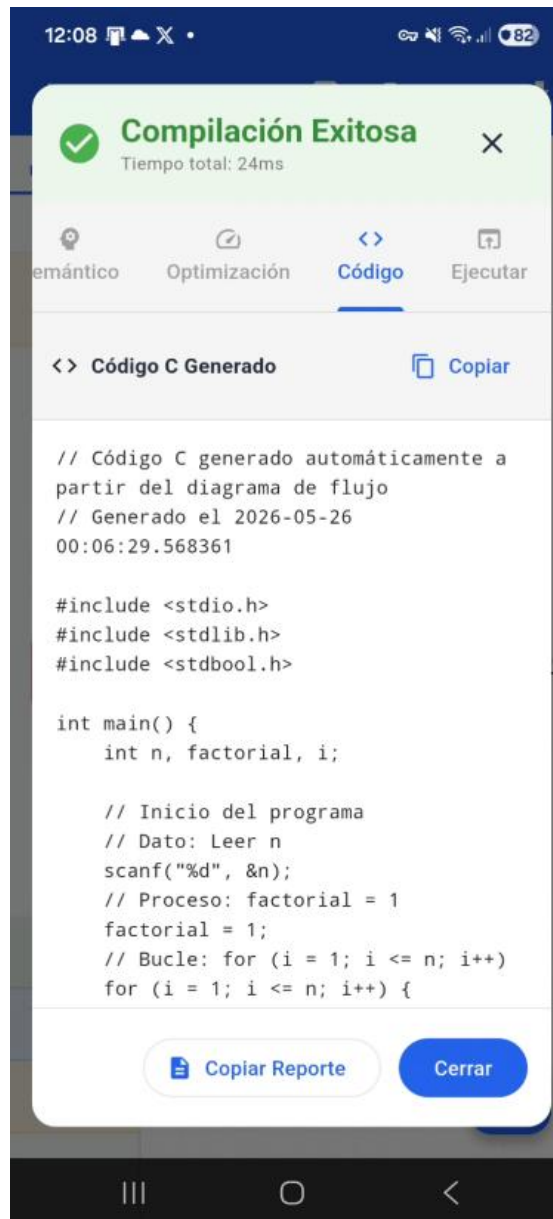
Copiar Reporte

Cerrar



RESULTADOS DEL OPTIMIZADOR

- **Evidencia 1.3 - Código C Generado:** Se exhibe el texto fuente en lenguaje C emitido por el compilador, evidenciando el mapeo correcto del bucle y la estructura general del archivo.



CÓDIGO C GENERADO

El código generado es:

```
// Código C generado automáticamente a partir del diagrama de flujo
```

```
// Generado el 2026-05-26 00:06:29.568361
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

#include <stdbool.h>

int main() {
    int n, factorial, i;

    // Inicio del programa

    // Dato: Leer n
    scanf("%d", &n);

    // Proceso: factorial = 1
    factorial = 1;

    // Bucle: for (i = 1; i <= n; i++)
    for (i = 1; i <= n; i++) {
        // Proceso: factorial = factorial * i
        factorial = factorial * i;
    }

    // Dato: Escribir "Factorial:", factorial
    printf("Factorial: %d\n", factorial);

    return 0;
}

```

Plantilla '14. Búsqueda Secuencial'

- **Evidencia 2.1 - Diagrama Base:** Se expone la estructura visual de la búsqueda secuencial en arreglos, mostrando el manejo de nodos condicionales e iterativos en la aplicación.

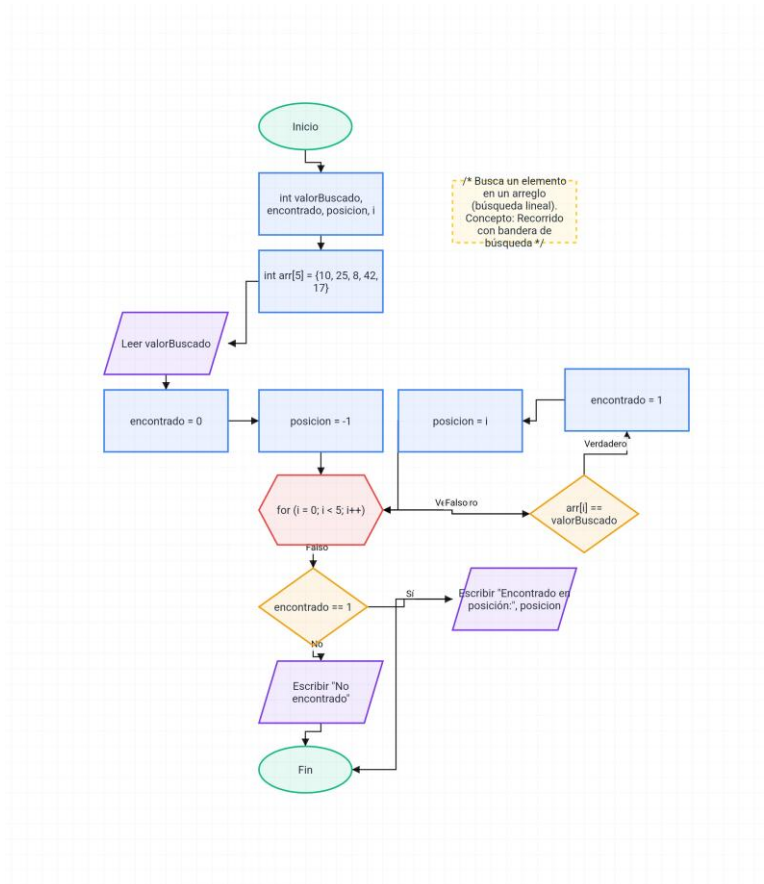
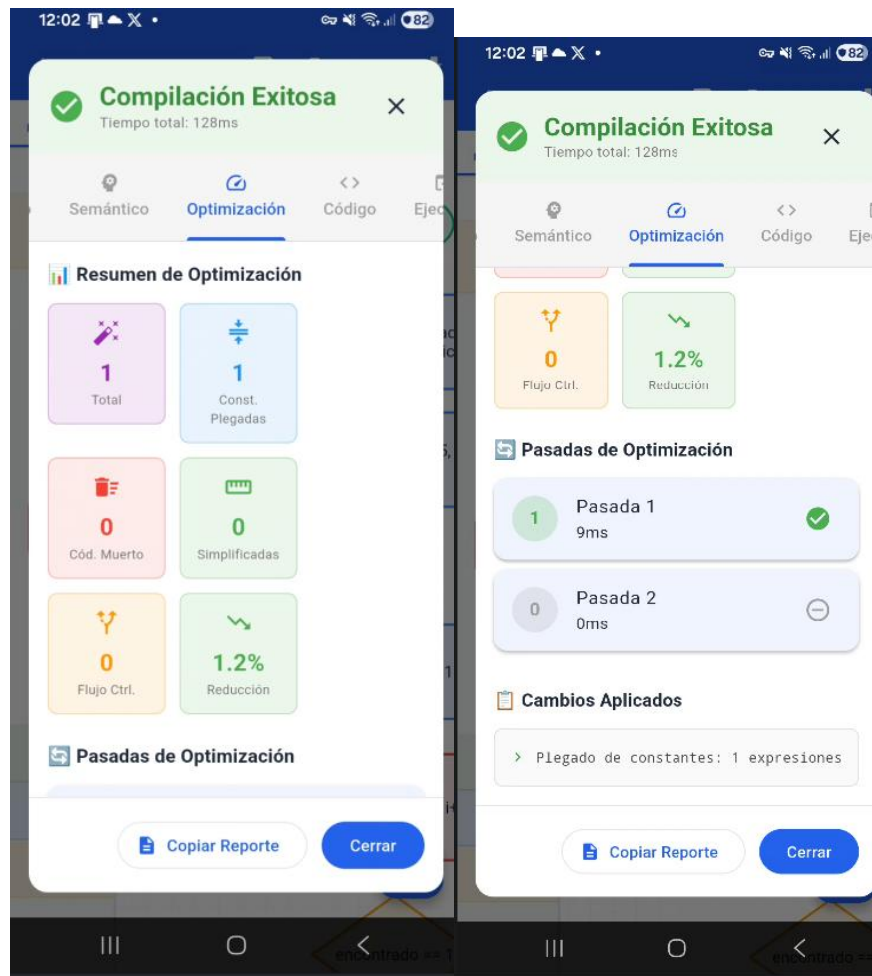


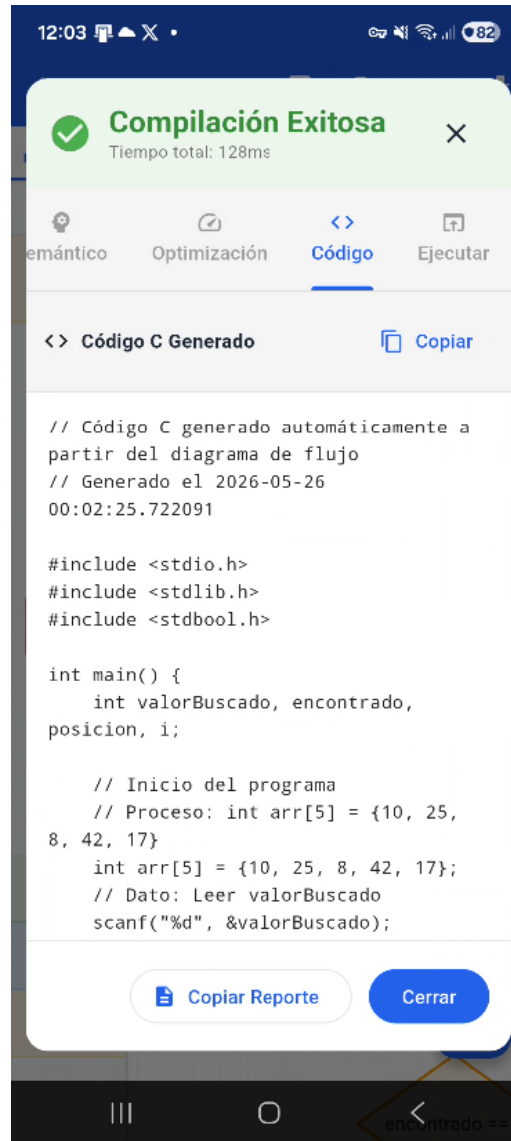
DIAGRAMA BÚSQUEDA SECUENCIAL

- **Evidencia 2.2 - Optimización del AST:** Se visualizan los resultados del proceso de optimización, destacando la simplificación de condiciones redundantes en el proceso de búsqueda del valor objetivo.



RESULTADOS DEL OPTIMIZADOR

- **Evidencia 2.3 - Código C Generado:** Se comprueba la correcta traducción de los accesos a arreglos y condiciones lógicas al estándar C99, incluyendo especificaciones de formato automático en las funciones printf/scanf.



CÓDIGO C GENERADO

El código generado es:

```
// Código C generado automáticamente a partir del diagrama de flujo
```

```
// Generado el 2026-05-26 00:02:25.722091
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <stdbool.h>
```

```
int main() {
```

```
    int valorBuscado, encontrado, posicion, i;
```

```
    // Inicio del programa
```

```
    // Proceso: int arr[5] = {10, 25, 8, 42, 17}
```

```
    int arr[5] = {10, 25, 8, 42, 17};
```

```
    // Dato: Leer valorBuscado
```

```
    scanf("%d", &valorBuscado);
```

```
    // Proceso: encontrado = 0
```

```
    encontrado = 0;
```

```
    // Proceso: posicion = -1
```

```
    posicion = -1;
```

```
    // Bucle: for (i = 0; i < 5; i++)
```

```
    for (i = 0; i < 5; i++) {
```

```
        // Decisión: arr[i] == valorBuscado
```

```
        if (arr[i] == valorBuscado) {
```

```
            // Proceso: encontrado = 1
```

```
            encontrado = 1;
```

```
            // Proceso: posicion = i
```

```
            posicion = i;
```

```
        }
```

```
    }
```

```
    // Decisión: encontrado == 1
```

```
    if (encontrado == 1) {
```

```
// Dato: Escribir "Encontrado en posición:", posicion
printf("Encontrado en posición: %d\n", posicion);
}
else {
    // Dato: Escribir "No encontrado"
    printf("No encontrado\n");
}

return 0;
}
```